

Cómo funciona la app y cómo cambiar los modelos 3D y los marcadores QR

AR Diabetes Tipo 1 — referencia para el equipo que va a personalizar el contenido: qué modelo va con qué página, qué se puede reemplazar arrastrando un archivo, y qué cambio ya necesita tocar código.

● Libro Fisiológico

● Libro Nutricional

● Libro Clínico

Documento de referencia técnica

3 modelos 3D · 12 marcadores QR · 1 archivo de contenido

Endify

2026-07-19

1. Qué es esto y para quién es

Esta guía es para quien vaya a tocar el **contenido** de la app: qué modelo 3D aparece en cada página, y qué imagen QR dispara ese modelo. No cubre diseño de UI ni las pantallas del menú — eso vive en otro lado del código y no hace falta para lo que se pide acá.

Hay dos niveles de cambio:

- **Reemplazar un archivo existente** (un modelo 3D, una imagen QR) — no requiere saber programar, solo reemplazar el archivo con el mismo nombre y volver a generar la app.
- **Agregar algo nuevo** (un libro extra, un marcador extra, un modelo que no se recolorea) — esto sí requiere a alguien que sepa C# básico y tenga el proyecto de Unity abierto. En cada sección se marca exactamente qué archivo y qué línea tocar.

2. Cómo funciona, en resumen

El contenido está organizado en **3 libros**. Cada libro tiene **un solo modelo 3D** y **un solo color**, y ese mismo modelo aparece en **4 páginas distintas** (4 marcadores QR) dentro de ese libro. Apuntar la cámara a cualquiera de esas 4 páginas hace aparecer el mismo modelo, ya escalado, girando solo, sobre la hoja — cambia el texto/tema de la pantalla, no el modelo.

Esto es lo primero para tener claro antes de pedir un cambio: **cambiar el modelo de un libro cambia las 4 páginas de ese libro a la vez**. Si lo que se quiere es un modelo distinto por página individual, hace falta reestructurar el contenido (ver sección 4).

LIBRO	MODELO 3D	COLOR	PÁGINAS / QR
● Fisiológico	Pancreas.fbx	#C86B6B	qr_diabetes · qr_pancreas · qr_insulina · qr_funciona
● Nutricional	Plato.fbx	#7CB86F	qr_nutri_plato · qr_nutri_carbohidratos · qr_nutri_recomendados · qr_nutri_habitos
● Clínico	Gluco.fbx	#4E8FD1	qr_clinico_insulina · qr_clinico_sintomas · qr_clinico_monitoreo · qr_clinico_cuidados

Todo esto — títulos, colores, textos, qué modelo va con qué libro — vive en un único lugar del código: `Assets/Scripts/AppBootstrap.BookContent.cs`, función `BuildBooks()`. Es el archivo a mirar si algo no coincide con esta tabla más adelante.

(Antes vivía adentro de `AppBootstrap.cs`, que tenía más de 2000 líneas con todas las pantallas juntas. Ese archivo se partió en varios más chicos por tema — este contenido ahora tiene un archivo propio y corto, más fácil de encontrar.)

3. Reemplazar un modelo 3D existente

Este es el cambio simple: no toca código.

- 1 Preparar el archivo nuevo.** Formato recomendado: **FBX** (es el que usan los 3 modelos actuales). Unity también importa OBJ y glTF, pero FBX es lo ya probado en esta app.
- 2 Reemplazar el archivo** en `Assets/Imagenes/QR/`, con el **mismo nombre exacto** del que va a reemplazar: `Pancreas.fbx`, `Plato.fbx` o `Gluco.fbx`. Unity lo vuelve a importar solo la próxima vez que se abra el proyecto.
- 3 Volver a generar la app** (ver sección 6) e instalarla en un celular para probar.

Dos cosas que la app ya resuelve sola

- **Escala:** no importa el tamaño real del archivo 3D — la app mide el modelo y lo reduce o agranda automáticamente a un tamaño estándar (~9 cm) para que se vea bien parado sobre la hoja. No hay que ajustar nada a mano.
- **Postura:** el modelo queda de pie, perpendicular a la página (no acostado), y gira solo sobre su propio eje.

IMPORTANTE — EL COLOR

La app pinta **todo** el modelo con un color parejo (la columna "Color" de la tabla de arriba), para que combine con la identidad de cada libro. Esto significa que cualquier textura o color propio que traiga el archivo 3D nuevo **no se va a ver** — queda tapado por ese color plano. Si el modelo nuevo necesita conservar sus texturas originales, es un cambio de una línea de código (quitar el bloque que reemplaza el material, en `ARController.cs` y `ModelViewer.cs`) — pedirlo así directamente a quien programe.

4. Agregar un modelo nuevo o cambiar de nombre de archivo

Esto sí requiere abrir el proyecto en Unity y tocar C# — son cambios chicos y puntuales:

- `Assets/Editor/ProjectSetup.cs`, líneas 57-59: acá se cargan los 3 archivos FBX por nombre. Si el archivo se llama distinto, se actualiza la ruta acá.
- `Assets/Scripts/AppBootstrap.BookContent.cs`, función `BuildBooks()` (línea 11 en adelante): cada bloque `books[i] = new BookDef { ... }` define el modelo, el color, el título y los 4 temas de un libro. Para un libro/modelo nuevo del todo, hay que sumar un cuarto bloque siguiendo el mismo patrón — pero eso ya es un cambio de alcance mayor (una cuarta sección completa de la app), no solo "cambiar un modelo".

Después de este tipo de cambio, hace falta recompilar (sección 6) para verlo reflejado.

5. Reemplazar o agregar un marcador QR

El "marcador" es la imagen impresa que la cámara reconoce para saber qué modelo mostrar.

REGLA DE ORO — EL TAMAÑO FÍSICO

El sistema de reconocimiento asume que el marcador impreso mide exactamente **12 × 12 cm**. Ese número está fijo en el código (no se puede cambiar por marcador individual) y lo usa para calcular a qué distancia real está la cámara — si se imprime a otro tamaño, el modelo puede aparecer con la escala incorrecta o con el anclaje inestable/tembloroso.

Imprimir siempre **a tamaño real (100%)**, nunca con "ajustar a la página". Ya existe una plantilla lista con los 12 marcadores calibrados a este tamaño exacto:

`Marcadores/marcadores_12_calibrados_12cm.pdf` .

Reemplazar un marcador existente

Mismo mecanismo que el modelo 3D: reemplazar el archivo PNG en `Assets/Markers/` con el mismo nombre exacto (ver la columna "Páginas / QR" de la tabla de la sección 2), y volver a generar la app. Formato: PNG cuadrado, 800×800 px es lo que usan los 12 actuales.

Agregar un marcador nuevo

Requiere código, en dos lugares:

- `Assets/Editor/ProjectSetup.cs`, líneas 92-94: ahí se lista, por libro, el nombre de archivo de cada uno de los 12 marcadores.
- `AppBootstrap.BookContent.cs`, dentro del bloque del libro correspondiente en `BuildBooks()`: el arreglo `Markers` y el `TopicTitle` asociado a esa página nueva.

Para que el reconocimiento salga bien

- Imágenes con buen detalle y sin simetría (los 12 QR actuales funcionan bien por eso) — evitar imágenes muy lisas, muy repetitivas o con simetría de espejo.

- Imprimir en papel mate — el papel brillante genera reflejos que le cuestan a la cámara.
- Escanear con buena luz, evitando sombras fuertes sobre la hoja.

6. Generar la app actualizada

Resumen rápido (el detalle completo, con los comandos exactos, está en el `README.md` del proyecto):

1. Abrir el proyecto en **Unity Hub** con el editor **6000.5.2f1**.
2. `File ▸ Build Profiles ▸ Android ▸ Build And Run`, con el celular conectado por USB y depuración USB activada.

Para automatizar (sin abrir la interfaz de Unity), el README trae el comando de línea de comandos equivalente, el mismo que se usó para todos los cambios de esta guía.

7. Para tener en cuenta

- **Peso de los modelos:** los 3 archivos actuales pesan entre 18 y 28 MB. Modelos más pesados (más polígonos, texturas grandes) hacen que la app tarde más en abrir y puede notarse menos fluida — conviene mantenerse en ese orden de tamaño o exportar una versión optimizada del modelo antes de sumarlo.
- **Probar siempre en un celular real** apuntando al marcador impreso — el editor de Unity no reproduce el reconocimiento de cámara real.
- **Guardar una copia** de los archivos originales (modelos y QR) antes de reemplazarlos, por si hay que volver atrás.